OpenFlow

A self-hosted Wispr Flow alternative

Hindi-English dictation · Custom dictionary · Claude-powered cleanup

Build time	Stack	Monthly cost	Replaces
~30 hours	**Python 3.10+**	**~$1.50**	**Wispr Flow Pro**

PREPARED FOR
Claude Code — autonomous build agent

VERSION
v1.0 — Spec Freeze

Build incrementally. Test after every phase.

SECTION 01

Project Overview

Goal

Build a free, self-hosted desktop dictation tool that replicates Wispr Flow's full feature set, with first-class support for **Hindi-English code-switching (Hinglish)** and a **user-extensible custom dictionary**.

Why

Wispr Flow charges $12–15 per month. We have a Claude API key, local compute, and a willingness to spend ~30 hours building. This pays for itself in two months and gives us full control over the stack, models, and dictionary.

Non-negotiables

- Hold-to-talk and tap-to-toggle hotkeys, fully global (works in any app)
- Sub-2-second latency from key release to text pasted
- Hindi → English translation mode AND mixed Hinglish transcription mode
- Custom dictionary that biases the transcriber toward user-specified terms
- Works offline for transcription; only needs internet for AI cleanup
- Cross-platform: macOS first, Linux second, Windows third

SECTION 02

Feature Parity Checklist (vs. Wispr Flow)

Build in order. Tick off as you go.

Tier 1 — Core Loop (MVP)

- ■ Global hotkey listener (hold-to-talk: fn or F5)
- ■ Audio capture from default mic at 16 kHz mono
- ■ Local Whisper transcription via faster-whisper
- ■ Claude API cleanup pass (remove fillers, fix grammar)
- ■ Auto-paste into focused field via clipboard + simulated paste
- ■ System tray icon with status (idle / recording / processing)
- ■ Visual + audio feedback on record start/stop

Tier 2 — Wispr Flow Power Features

- ■ **Tap-to-toggle mode** in addition to hold-to-talk
- ■ **Auto-formatting**: punctuation, capitalization, paragraph breaks
- ■ **Tone modes**: Casual / Professional / Bullet Points / Email / Slack
- ■ **Context awareness**: detect active app and adjust tone
- ■ **Edit mode**: select text, hit hotkey, dictate edit instruction → AI rewrites
- ■ **Undo last paste** hotkey
- ■ **Dictation history**: last 50 transcriptions, searchable
- ■ **Settings GUI** (PyQt or web-based via local server)

Tier 3 — Hindi/Hinglish Support *(THE differentiator)*

- Language mode switcher: EN / HI / HINGLISH / AUTO
- Hinglish transcription: keep Hindi words in Devanagari OR Roman, user-toggleable
- Hindi-to-English translation mode (speak Hindi, get English text)
- English-to-Hindi translation mode (speak English, get Hindi text)
- Code-switching detection (sentences mixing both languages)
- Per-mode hotkey or rotating hotkey

Tier 4 — Custom Dictionary *(THE other differentiator)*

- User dictionary file (~/.openflow/dictionary.json)
- Whisper initial_prompt injection for transcription bias
- Post-transcription fuzzy correction (e.g., "Oh la flock" → "Oltaflock")
- Phonetic mapping for Indian names/places
- CLI to add/remove/list dictionary entries
- GUI dictionary editor

Tier 5 — Polish

- Auto-launch on login (LaunchAgent / systemd / Task Scheduler)
- Onboarding wizard (mic test, hotkey config, API key setup)
- Crash recovery + telemetry-free error logging
- Update checker
- Export/import settings

SECTION 03

Architecture

Single Python process running in the system tray. All modules talk through the central **Daemon** orchestrator. Single source of truth for config and state lives in ~/.openflow/.

Component	Responsibility
HotkeyListener	Captures global key events (pynput), fires start/stop
AudioRecorder	Streams 16 kHz mono PCM from selected input device
Transcriber	faster-whisper inference; loads dictionary as initial_prompt
DictionaryCorrector	Fuzzy-matches output against user dictionary (rapidfuzz)
AIProcessor	Calls Claude Haiku for cleanup, translation, edit-mode rewrites
Paster	Writes to clipboard + triggers OS-native paste keystroke
TrayUI	Status icon, mode switcher, history, settings entry point
Daemon	Wires everything; holds shared state; owns config + lifecycle

STATE: *Single source of truth: ~/.openflow/config.toml for settings, dictionary.json for terms, history.sqlite for past dictations.*

SECTION 04

Tech Stack

Layer	Library	Why
Audio capture	sounddevice + numpy	Reliable, cross-platform, low-latency
Speech-to-text	faster-whisper	4x faster than openai-whisper, good Hindi
Hotkeys	pynput	Cross-platform global hotkeys
Clipboard	pyperclip	Battle-tested
Paste sim	osascript / xdotool / pyautogui	Native is most reliable
AI cleanup	anthropic SDK	Claude Haiku 4.5 for speed + cost
Tray UI	pystray + Pillow	Simple, works everywhere
Settings GUI	PyQt6 OR Flask + browser	PyQt heavier but more native
Fuzzy matching	rapidfuzz	Fast Levenshtein for dictionary correction
Config	tomli / tomli-w	TOML is more user-friendly than JSON
History DB	sqlite3 (stdlib)	No setup
Packaging	pyinstaller	One-file binary for distribution

Models

- Whisper: small for English, medium for Hindi/Hinglish — Hindi accuracy is much better at medium+
- Claude: claude-haiku-4-5-20251001 for cleanup (fast, cheap, ~$1/M input tokens)

SECTION 05

Project Structure

```
openflow/
■■■ README.md
■■■ PROJECT_PLAN.md
■■■ pyproject.toml
■■■ requirements.txt
■■■ .env.example
■■■ openflow/
■ ■■■ __init__.py
■ ■■■ __main__.py # entrypoint
■ ■■■ daemon.py # main orchestrator
■ ■■■ config.py # load/save TOML config
```

```
■ ■■■ hotkeys.py # pynput hotkey manager
■ ■■■ audio.py # recorder
■ ■■■ transcribe.py # faster-whisper wrapper
■ ■■■ dictionary.py # custom term management + fuzzy correction
■ ■■■ ai.py # Claude API calls
■ ■■■ paste.py # OS-specific paste logic
■ ■■■ tray.py # pystray icon + menu
■ ■■■ history.py # sqlite log
■ ■■■ prompts.py # all system prompts in one place
■ ■■■ ui/
■ ■■■ settings.py # PyQt settings window
■ ■■■ dict_editor.py # dictionary GUI
■ ■■■ onboarding.py # first-run wizard
■■■ tests/
■ ■■■ test_dictionary.py
■ ■■■ test_ai.py
■ ■■■ fixtures/
■ ■■■ sample_audio.wav
■■■ scripts/
■■■ install_macos.sh
■■■ install_linux.sh
```

SECTION 06

Phase-by-Phase Implementation

Phase	Duration	Output
0 — Bootstrap	30 min	venv + dependencies installed, structure scaffolded
1 — Core Loop	Day 1	Hotkey → record → transcribe → paste working end-to-end
2 — Cleanup + Tones	Day 1 PM	Claude integration, 5 tone modes, mode hotkeys
3 — Tray UI	Day 2 AM	pystray icon, status states, mode/language menus
4 — Hindi/Hinglish	Day 2 PM	All 6 language modes (EN, HI, HI_ROMAN, HINGLISH, HI→EN, EN→HI)
5 — Custom Dictionary	Day 3	JSON dict, Whisper biasing, fuzzy correction, CLI
6 — Edit Mode	Day 4	Select text → speak instruction → AI rewrites in place
7 — Settings GUI	Day 5	PyQt6 window, all tabs functional
8 — Packaging	Day 6	One-file binary, auto-launch installer

Phase 0 — Bootstrap

```
mkdir openflow && cd openflow
python -m venv .venv && source .venv/bin/activate
pip install faster-whisper sounddevice numpy pyperclip pynput \
anthropic scipy pystray pillow rapidfuzz tomli tomli-w \
PyQt6 keyring
pip freeze > requirements.txt
```

Phase 1 — Core Loop (build in this exact order)

- **audio.py** — Recorder class with start() / stop() → np.ndarray. Test by recording 3s and saving WAV.
- **transcribe.py** — Transcriber wrapping faster-whisper. Test on a known audio file.
- **paste.py** — paste(text) function: clipboard write + Cmd+V / Ctrl+V keystroke.
- **hotkeys.py** — Listener firing on_press_start and on_release_stop callbacks.
- **daemon.py** — wires it all together.

CHECKPOINT: *Holding F5 anywhere on your machine should dictate raw transcription into the focused text field before you move on. Don't proceed until this works.*

Phase 2 — Claude Cleanup + Tone Modes

Define system prompts for each tone in prompts.py: **CASUAL, PROFESSIONAL, BULLETS, EMAIL, SLACK, RAW**. Implement ai.cleanup(text, mode, context_app=None) using Haiku. Wire mode-switching hotkeys (Cmd+Shift+1 through 5).

Sample system prompt — PROFESSIONAL

```
You are a dictation cleanup assistant. Clean up the user's voice transcription:
- Remove filler words (um, uh, like, you know, basically)
- Fix grammar and punctuation
- Maintain the speaker's intent and voice
- Output professional but not stiff prose
- Return ONLY the cleaned text, no preamble, no quotes, no commentary
```

Phase 4 — Hindi / Hinglish Support *(critical)*

This is where most projects fail. Whisper's Hindi support is good but needs careful configuration. Read this section twice before implementing.

- **Whisper language flag**: pass language="hi" for pure Hindi, language=None for auto-detect (Hinglish).
- **Model size**: Use medium or large-v3 for Hindi. base and small butcher it.
- **Output script**: Whisper outputs Hindi in Devanagari by default. To get Roman script ("typed Hinglish"), pass through Claude with a transliteration prompt.
- **Translation modes**: Whisper has built-in task="translate" that translates any source language to English. Use it for HI→EN.
- **Reverse translation (EN→HI)**: Whisper can't do this. Transcribe English, then call Claude: *"Translate this English text to Hindi in Devanagari."*

Mode matrix

Mode	Whisper language	Whisper task	Post-Claude action
EN	en	transcribe	Cleanup
HI	hi	transcribe	Cleanup (Devanagari)
HI_ROMAN	hi	transcribe	Transliterate to Roman
HINGLISH	None (auto)	transcribe	Cleanup, preserve mix
HI_TO_EN	hi	translate	Cleanup
EN_TO_HI	en	transcribe	Translate to Hindi via Claude

Add a **rotating hotkey** (e.g., F6) to cycle through modes, with tray notification on switch.

Phase 5 — Custom Dictionary

Solves the proper-noun problem (names, jargon, brand names like "Oltaflock", "Khaana").

Schema — dictionary.json

```
{
"terms": [
{
"canonical": "Oltaflock",
"phonetic_hints": ["oh la flock", "ola flock", "olaf lock"],
"language": "en",
"context": "company name"
},
{
"canonical": "Bhopal",
"phonetic_hints": ["bopal", "bhopaal"],
"language": "both"
},
{
"canonical": "Khaana",
"phonetic_hints": ["kana", "khana", "khaanaa"],
"language": "both",
"context": "product name"
}
]
}
```

Whisper biasing

Build a string of canonical terms separated by commas, prefixed with: *"Glossary of terms that may appear: ..."*. Pass as initial_prompt to model.transcribe(). Whisper will be biased toward these spellings. Cap at ~200 tokens to avoid truncation.

Post-transcription fuzzy correction

```
from rapidfuzz import fuzz, process

def correct(text: str, dictionary: list[dict]) -> str:
words = text.split()
corrected = []
for word in words:
best_match, score, idx = process.extractOne(
word.lower(),
[h for entry in dictionary for h in entry['phonetic_hints']],
scorer=fuzz.ratio
)
if score > 85:
canonical = next(
e['canonical'] for e in dictionary
if best_match in e['phonetic_hints']
)
corrected.append(canonical)
else:
corrected.append(word)
return ' '.join(corrected)
```

CLI

```
openflow dict add "Oltaflock" --hints "oh la flock,ola flock"
openflow dict list
openflow dict remove "Oltaflock"
```

Phase 6 — Edit Mode

Wispr Flow's killer feature: select text anywhere, hit hotkey, speak instruction, AI rewrites in place.

- Hotkey: Cmd+Shift+E

- On trigger: copy current selection (Cmd+C), record audio.
- Build Claude prompt with selection + instruction; paste result over selection.

```
You are an inline text editor. The user selected this text:
---
{selected_text}
---
And gave this instruction: "{transcribed_instruction}"

Apply the instruction and return ONLY the edited text. No preamble, no quotes.
```

SECTION 07

Configuration File

Lives at ~/.openflow/config.toml. Generated on first run by the onboarding wizard. Editable by hand or via the Settings GUI.

```
[general]
auto_launch = true
default_tone = "professional"
default_language = "auto"
hindi_script = "devanagari" # or "roman"

[hotkeys]
record_hold = "f5"
record_toggle = "cmd+shift+space"
edit_mode = "cmd+shift+e"
cycle_mode = "f6"
undo_paste = "cmd+shift+z"

[audio]
sample_rate = 16000
device = "default"
silence_threshold = 0.01

[whisper]
model = "medium" # tiny / base / small / medium / large-v3
device = "cpu" # cpu / cuda / mps
compute_type = "int8" # int8 for CPU, float16 for GPU

[claude]
model = "claude-haiku-4-5-20251001"
max_tokens = 1024
api_key_env = "ANTHROPIC_API_KEY"

[dictionary]
fuzzy_threshold = 85
inject_into_whisper = true
```

SECTION 08

System Prompts Library — prompts.py

Key	Purpose
casual	Light cleanup, preserves conversational tone
professional	Filler removal + grammar fixes, professional but natural
bullets	Convert dictation into clean bullet points
email	Format as email body, add greeting/sign-off only if context suggests
slack	Concise Slack message, no greetings

Key	Purpose
transliterate_hi_to_roman	Devanagari → Roman natural transliteration
translate_en_to_hi	English → Hindi (Devanagari), tone-preserving
edit_selection	Inline editor: applies instruction to selected text

Sample — transliterate_hi_to_roman

```
Convert this Hindi text written in Devanagari to natural Roman/Latin
transliteration as Indians type it on phones.
Examples:
■■■■■■■ → namaste
■■■ ■■ ■■ ■■■■ ■■■■ → main ghar ja raha hoon
Return ONLY the transliteration.
```

SECTION 09

Testing Strategy

Unit tests

- test_dictionary.py: fuzzy correction edge cases ("oltaflock", "ola flock", "Oltaflock Inc")
- test_ai.py: mock Anthropic client, verify prompt construction

Integration smoke tests

- Record: *"Hi, this is a test of Oltaflock and Khaana, based in Bhopal."* — expect all three custom terms spelled correctly.
- Hindi: *"■■■■ ■■ ■■■■■■■■ ■■ ■■■■ ■■■■■"* — HI mode: clean Devanagari; HI_TO_EN: "I am going to Bangalore tomorrow."
- Hinglish: *"Yaar I think we should ship the MVP by Friday, kya bolte ho?"* — preserves both languages.

Manual checklist before declaring "done"

- ■ 50 dictations across 5 different apps without crash
- ■ Hotkey works after Mac sleep/wake cycle
- ■ Latency < 2s for 10-second clips
- ■ "oh la flock" → "Oltaflock" 9 out of 10 times
- ■ Hindi mode produces correct Devanagari for common phrases

SECTION 10

Cost Model

Item	Per dictation	Per month (100/day)
Whisper local	$0	$0

Item	Per dictation	Per month (100/day)
Claude Haiku cleanup (~500 tok in/out)	~$0.0005	~$1.50
Total	~$0.0005	~$1.50

Wispr Flow Pro: $15/month → break-even immediate, savings of ~$160/year.

SECTION 11

Known Pitfalls (read carefully)

#	Issue	Mitigation
1	macOS permissions	Accessibility (for hotkey + paste) AND Microphone must be granted. The first-run wizard
2	Whisper Hindi quality drops with <code>small</code>	Always use <code>medium</code> minimum for Hindi. The 8x compute cost is worth it.
3	pynput on Wayland (Linux)	Doesn't work. Detect Wayland and fall back to <code>evdev</code> or warn the user.
4	Paste timing	Need ~100ms sleep between clipboard write and paste keystroke, or it pastes the previo
5	Audio device hot-swap	If user plugs in AirPods mid-session, sounddevice may break. Wrap recorder in retry logi
6	Long recordings	Whisper quality degrades past ~30s. Either chunk audio or warn the user.
7	Claude rate limits	Haiku has generous limits but free tier can hit them. Add exponential backoff.
8	Devanagari font in tray menu	pystray uses system fonts; on Linux without Indian fonts, Hindi menu items show as boxe
9	Custom dictionary token cost	Don't dump 500 terms into Whisper's <code>initial_prompt</code> — 224-token limit. Sn
10	Don't store API key in <code>config.toml</code>	Keep it in env var or OS keychain (<code>keyring</code> library).

SECTION 12

Stretch Goals (post-v1)

- **Voice activity detection (VAD)**: auto-stop on 1.5s silence (silero-vad)
- **Streaming transcription**: show partial results live
- **Multi-language dictionary**: per-language phonetic hints
- **Team sync**: shared dictionary via a Git repo
- **iOS companion** for dictating to your Mac
- **Local LLM cleanup** via Ollama (fully offline mode)
- **Custom wake word** ("Hey Flow")

SECTION 13

First Commands for Claude Code

When you start, run these in order:

```
# 1. Verify environment
python --version # Need 3.10+
which ffmpeg # Required by faster-whisper

# 2. Bootstrap project
mkdir -p openflow/openflow/ui openflow/tests/fixtures \
openflow/scripts openflow/assets
cd openflow

# 3. Set up venv and install
python -m venv .venv
source .venv/bin/activate
pip install faster-whisper sounddevice numpy pyperclip pynput \
anthropic scipy pystray pillow rapidfuzz tomli \
tomli-w PyQt6 keyring

# 4. Build Phase 1 (core loop) before anything else.
# Test it works end-to-end before moving on.
```

SECTION 14

Definition of Done

OpenFlow v1.0 ships when:

- A non-technical user can install via a single script and start dictating in 5 minutes.
- All Tier 1, 2, 3, 4 features work without manual intervention.
- It survives 1 week of daily personal use without crashing.
- Hindi/Hinglish accuracy is subjectively ≥ Wispr Flow's English accuracy.
- "Oltaflock", "Khaana", "Bhopal", "Bangalore" handled correctly 95%+ of the time.

Build incrementally. Test after every phase. Don't write Phase 5 code while Phase 1 is broken.